Rockchip 算法统一框架（ROCKX2）SDK开发指南

文件标识：RK-XX-XX-nnn

发布版本：V1.0.0

日期：2025-06-21

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

ROCKX算法框架统一各算法SDK的开发规范和接口调用，以及后续能够对各算法SDK包提供统一的下载、更新、安装管理，方便客户在RK芯片平台的集成算法功能。

产品版本

芯片名称	系统版本
RV1126B/RV1106B/RV1106	Linux
RK3588/RK3576/RK356X	Linux/Android

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V2.0.0	杨华聪	2025/06/21	初始文档

目录

Rockchip 算法统一框架（ROCKX2）SDK开发指南

- 概述
- 算法SDK开发说明
  - 算法SDK介绍
  - 算法SDK规范
  - 算法SDK调用
    - 初始化
    - 参数配置
    - 授权配置
    - 执行
    - 调用示例
  - 自定义模块
  - 自定义计算图
- API参考
  - 主要接口
    - 函数定义
      - RockXCreate
      - RockXDestroy
      - RockXLoadSDK
      - RockXInit
      - RockXSetParam
      - RockXGetParam
      - RockXSetNodeState
      - RockXSetInputInfo

- RockXSetOutputInfo
- RockXRemoveOutputInfo
- RockXInputCreate
- RockXInputDestroy
- RockXOutputCreate
- RockXOutputDestroy
- RockXProcess
- RockXProcessAsync
- RockXWaitFinish
- RockXInputPacketsNum
- RockXOutputPacketsNum
- RockXInputGetPacketAt
- RockXOutputGetPacketAt
- RockXInputGetPacketByName
- RockXOutputGetPacketByName

结构体定义

- RockXInput
- RockXOutput
- RockXPacket
- RockXParam
- RockXParamCallback
- RockXValue
- RockXMemoryBuffer
- RockXTensorQntInfo
- RockXTensorShape
- RockXTensor
- RockXImage
- RockXAudioWav
- RockXKeyValue

枚举定义

- RockXRetCode
- RockXPacketState
- RockXPacketMode
- RockXPacketType
- RockXDataType
- RockXNodeAction
- RockXModuleInstanceMode
- RockXCallbackMode
- RockXStreamState
- RockXMemType
- RockXTensorFormat
- RockXTensorQntType
- RockXPixelFormat
- RockXImageTransform
- RockXValueType

模块接口

函数定义

- RockXModuleRegister
- RockXModuleUnRegister
- RockXModuleGlobalRegister
- RockXModuleGlobalUnRegister
- RockXModuleSetExtData
- RockXModuleGetExtData

RockXModuleGetSharedDataAt
RockXModuleGetModule
RockXModuleGetParam
RockXModuleGetParamInt
RockXModuleGetParamFloat
RockXModuleGetParamString
RockXModuleGetParamPointer
RockXModuleGetParamAt
RockXModuleGetParamIntAt
RockXModuleGetParamFloatAt
RockXModuleGetParamStringAt
RockXModuleGetParamPointerAt
RockXModuleSetParam
RockXModuleAuthCheck
回调函数定义
OnModuleRegister
OnModuleUnRegister
OnModuleBuilder
OnModulePreInit
OnModuleInit
OnModulePostInit
OnModuleRelease
OnModuleParamSet
OnModuleParamGet
OnModuleInputGet
OnModuleOutputGet
OnModulePreProcess
OnModuleProcess
OnModulePostProcess
结构体定义
RockXModule
RockXModuleContext
枚举定义
RockXModuleType
RockXParamScope
计算图构建接口
函数定义
RockXBuilderGraphCreate
RockXBuilderGraphDestroy
结构体定义
RockXNodeInfo
RockXGraphInfo
工具函数
RockXPacket操作函数
RockXPacketSetInt
RockXPacketSetLong
RockXPacketSetDouble
RockXPacketSetFloat
RockXPacketSetString
RockXPacketSetPointer
RockXPacketSetTensor
RockXPacketSetImage
RockXPacketShareData

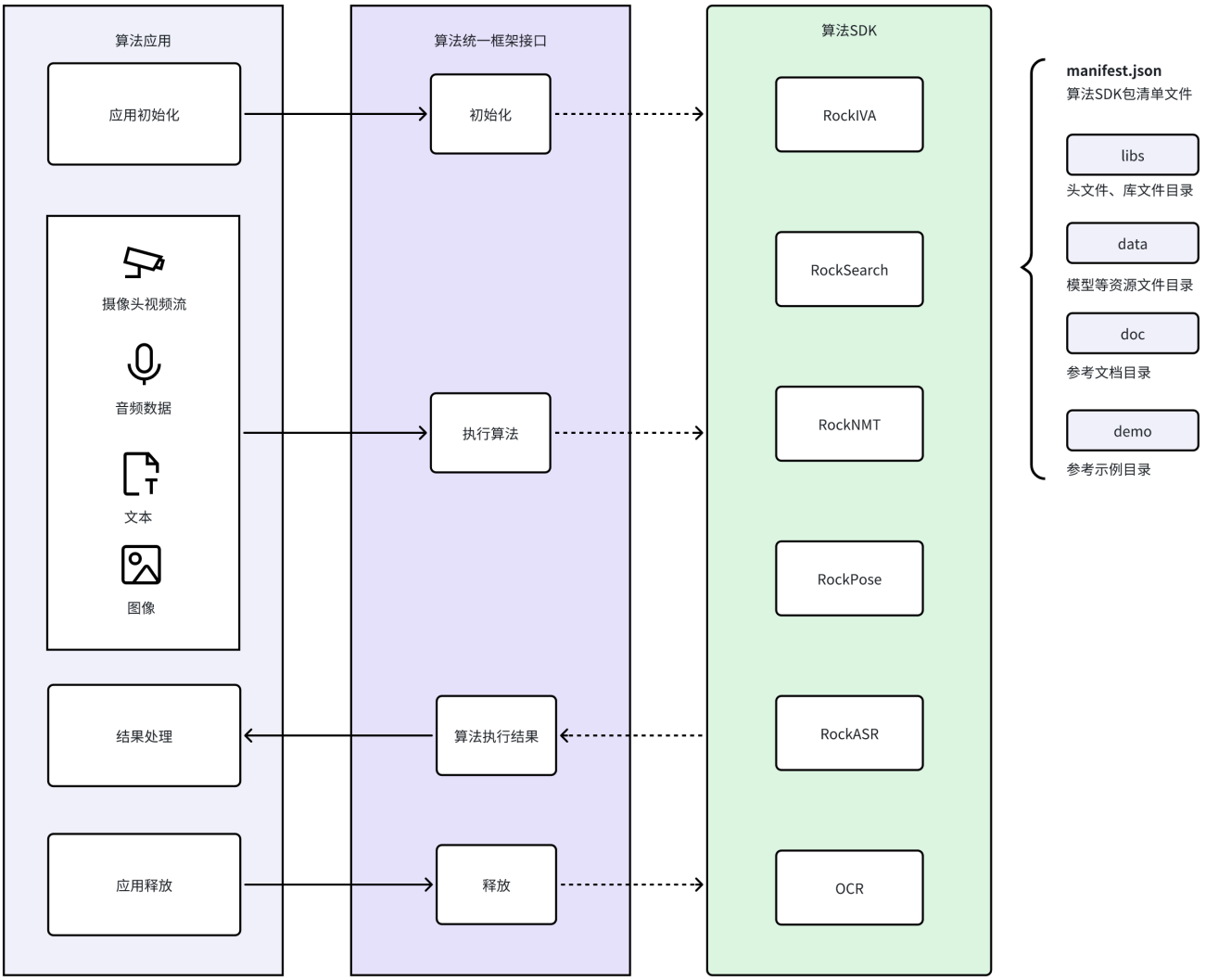
RockXPacketMoveData
RockXParam操作函数
RockXSetParamInt
RockXSetParamLong
RockXSetParamFloat
RockXSetParamString
RockXSetParamPointer
RockXParamCreate
RockXParamDestroy
RockXParamDup
RockXParamDup2
RockXValue操作函数
RockXValueSetInt
RockXValueSetLong
RockXValueSetDouble
RockXValueSetFloat
RockXValueSetString
RockXValueSetPointer
RockXValueSetArray
RockXMem操作函数
RockXMemAlloc
RockXMemFree
RockXMemSync
RockXMemFreeByVirtAddr
RockXMemDumpAll
RockXTensor操作函数
RockXTensorReadFromFile
RockXTensorWrite
RockXTensorWriteTxt
RockXTensorSize
RockXTensorDump
RockXTensorGetData
RockXTensorGetData1
RockXTensorGetDataInt8
RockXImage操作函数
RockXImageSize
RockXImageAllocBuffer
RockXImageFreeBuffer
RockXImageSyncBuffer
RockXImageWriteRaw

概述

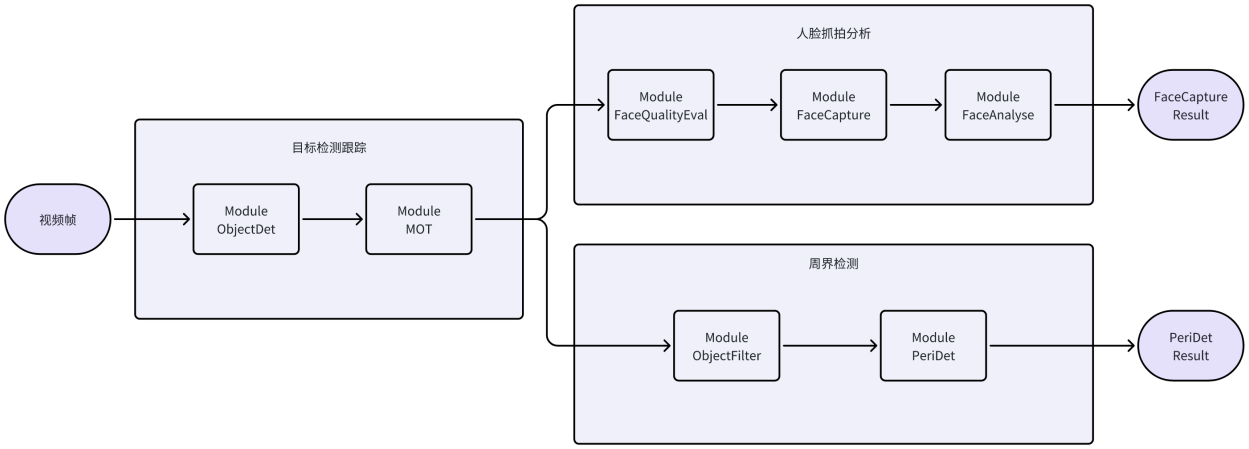
ROCKX算法框架统一各算法SDK的开发规范和接口调用，以及后续能够对各算法SDK包提供统一的下载、更新、安装管理，方便客户在RK芯片平台的集成算法功能。

主要功能有：

- 提供统一的C API接口，应用通过统一接口对算法SDK进行加载，通过统一的接口对算法SDK中的算法任务/模块进行调用



- 算法SDK通过模块/计算图提供调用
  - 可以支持单独调用模块/计算图，可以支持多个算法SDK的中的算法模块和计算图组合使用
  - 计算图可以支持组合嵌套使用，可以将一个开发完成的计算图作为子图嵌套到另外一个算法任务的计算图中
  - 支持用户加入自定义节点，算法任务的流程可以支持加入应用自定义模块到流程中，满足应用自定义需求



- 内置常用算法模块，可以开发自定义算法模块

算法SDK开发说明

算法SDK介绍

算法SDK为一个算法方案的集合，可以提供多个算法任务或算法模块给用户进行调用，算法SDK内使用模块和计算图来提供接口：

- 模块：模块实现单一的算法功能，可以单独调用，也可以组合到计算图中
- 计算图：计算图表示一个算法任务，计算图由多个模块节点组成，节点之间有数据依赖关系，计算图可以作为子图嵌套使用

算法SDK规范

为了能够方便框架进行加载，每个算法SDK遵守统一的规范。每个算法SDK包目录结构如下：

- manifest.json：算法包清单文件，描述SDK提供的模块、算法任务等接口信息
- libs：算法SDK的头文件、算法库和依赖库
- data: 模型文件和资源文件
- demo: 参考示例
- doc: 参考文档

manifest.json内容示例：

```
{
    "name": "rocktest",
    "version": "1.0.0",
    "description": "A simple RockX SDK for test and reference",
    "author": "Rockchip",
    "libs" : {
        "rocktest": {
            "soname": "librocktest.so",
            "graphs": [
                {
                    "name": "Calculation",
                    "description": "Calculation out = (in1 + in2 - in2) - in3",
                    "inputs": {
                        "in1": {
                            "type": "RockXValue",
                            "description": "Input 1"
                        },
                        "in2": {
                            "type": "RockXValue",
                            "description": "Input 2"
                        },
                        "in3": {
                            "type": "RockXValue",
                            "description": "Input 3"
                        }
                    },
                    "outputs": {
                        "out": {
```

```
                            "type": "RockXValue",
                            "description": "Output"
                        }
                    }
                }
            ],
            "modules": [
                {
                    "name": "Mul",
                    "description": "Mul out = in1 x in2",
                    "inputs": {
                        "in1": {
                            "type": "RockXValue",
                            "description": "Input 1"
                        },
                        "in2": {
                            "type": "RockXValue",
                            "description": "Input 2"
                        }
                    },
                    "outputs": {
                        "out": {
                            "type": "RockXValue",
                            "description": "Output"
                        }
                    }
                }
            ]
        }
    },
    "configs": {
        "test_config": 1
    }
}
```

- name：算法SDK名称
- version：算法SDK版本
- description：SDK介绍描述信息
- author：作者信息
- libs：算法库文件
    - soname：库名称（so库需要放在libs目录下）
    - graphs：算法SDK对外提供的算法任务函数接口，例如人脸抓拍、周界、车辆车牌识别等算法任务，每个graph包含如下的字段：
        - configs：图支持的配置选项
        - inputs：图的输入
        - outputs：图的输出
    - modules：算法SDK对外提供的单独模块，可以让用户单独调用，例如目标检测、人脸质量、目标跟踪等，每个module包含如下字段：

    - configs：模块支持的配置选项
    - inputs：模块的输入
    - outputs：模块的输出
- configs：SDK的全局变量，加载SDK后会被加入全局变量，可以通过`<sdk_name>.<config_name>`访问，例如`rocktest.`test_config，应用代码通过接口可以对config进行读写。

算法SDK调用

算法SDK包含了一系列算法任务和模块的集合，应用可以通过统一的接口来对算法SDK进行加载，对算法SDK中的算法任务和模块进行调用。

初始化

- 创建实例

创建一个实例

```
RockXHandle handle = RockXCreate(0, NULL);
```

如果要异步执行，需要设置一个异步标志，如下

```
RockXHandle handle = RockXCreate(ROCKX_FLAG_ASYNC_EXECUTE_MASK, NULL);
```

- 加载算法SDK

```
ret = RockXLoadSDK(handle, "/data/rocktest");
```

- 配置要调用的模块

如果要调用模块

```
RockXSetParamString(handle, NULL, "module_name", "Mul", 0);
```

- 初始化

```
ret = RockXInit(handle);
```

参数配置

通过RockXSetParam或RockXSetParamXXX接口可以设置全局或某个节点的参数

示例如下：

```
# 配置全局参数
RockXSetParamString(handle, NULL, "root_node_name", "iva_det", 0);
RockXSetParamString(handle, NULL, "module_name", "IvaDet", 0);
# 配置节点参数
RockXSetParamFloat(handle, "iva_det.det_result_filter", "threshold", 0.5f);
```

- 第一个参数为创建的实例句柄

- 第二个参数为节点名

  使用<root_node_name>.x.y.z形式的节点名来表示嵌套图的子节点，其中`root_node_name`为加载的模块名/计算图名称

  root_node_name可以使用参数如果没有指定，默认使用模块名/计算图的类型作为根节点名称

  如果节点名设为NULL，表示设置一个全局参数

- 第三个参数为参数名称

  当前SDK内置支持的一些全局配置的名如下：

配置名	说明
root_node_name	设置根节点名称
module_name	设置要调用模块名称
graph_name	设置要调用的计算图名称
graph_info	设置要调用的计算图
data_path	资源文件目录路径
auth_config	授权配置文件路径

授权配置

授权配置文件内容如下，可以分别指定需要授权的模块的License信息，其他详细配置信息可以参考对应sdk的`rkauth`目录下的文档`Rockchip_User_Guide_RKAUTH_CN`

```
{
    "rkauth_modules_config": [
        {
        "module": "nmt",
        "license_path": "/sdcard/key.lic"
        },
        {
        "module": "iva",
        "license_path": "/sdcard/key.lic"
        },
    ]
}
```

执行

- **输入数据**

输入数据用RockXInput表示

通过调用RockXInputCreate函数可以创建一个RockXInput对象，调用RockXInputDestroy可以进行释放。

其中通过[RockXInputPacketAt]可以从RockXInput中获取数据包，数据包的索引和模块/计算图定义的输入顺序保持一致。

- **输出数据**

输出数据用RockXOutput表示

通过调用RockXOutputCreate可以创建一个RockXOutput对象，调用RockXOutputDestroy可以进行释放。

其中通过[RockXOutputPacketAt]可以从RockXOutput中获取数据包，数据包的索引和模块/计算图定义的输出顺序保持一致。

- **数据包设置和读取**

  输入输出数据包的中的数据可以通过RockXPacketSetXXX和RockXPacketGetXXX等函数进行设置

- **执行**

执行方式有两种

- 同步执行：调用RockXProcess会阻塞直至执行完成才会返回
- 异步执行： 调用RockXProcessAsync，提交输入后立即返回，执行结果通过回调函数返回

调用示例

调用模块示例

```
    // 创建实例
    RockXHandle handle = RockXCreate(0, NULL);
    // 加载SDK
    RockXLoadSDK(handle, "/data/rocktest");
    // 指定模块名
    RockXSetParamString(handle, NULL, ROCKX_KEY_MODULE_NAME, "Mul", 0);
    // 初始化
    RockXInit(handle);

    // 准备输入输出
    const char* out_names[1] = {
        "out"
    };

    RockXInput* input = RockXInputCreate(handle);
    RockXOutput* output = RockXOutputCreate(handle, out_names, 1);

    RockXPacket* in1 = RockXInputGetPacketAt(input, 0);
    RockXPacket* in2 = RockXInputGetPacketAt(input, 1);

    RockXPacketSetInt(in1, 1);
    RockXPacketSetInt(in2, 2);

    // 执行
    RockXProcess(handle, input, output);

    // 获取输出
    RockXPacket* packet_objects = RockXOutputGetPacketAt(output, 0);
    if (packet_objects == NULL) {
        printf("Can't find output packet\n");
        return -1;
```

```
    }
    int v = RockXPacketGetInt(packet_objects, -1);
    printf("output: %d\n", v);

    // 销毁释放
    RockXInputDestroy(handle, input);
    RockXOutputDestroy(handle, output);

    RockXDestroy(handle);
```

调用计算图示例

```
    // 创建实例
    RockXHandle handle = RockXCreate(0, NULL);
    // 配置计算图
    RockXGraphInfo* graph = BuildGraph();
    RockXSetGraphInfo(handle, graph);
    // 初始化
    RockXInit(handle);

    // 准备输出
    const char* out_names[1] = {
        "out"
    };
    RockXInput* input = RockXInputCreate(handle);
    RockXOutput* output = RockXOutputCreate(handle, out_names, 1);

    RockXPacket* in1 = RockXInputGetPacketAt(input, 0);
    RockXPacket* in2 = RockXInputGetPacketAt(input, 1);
    RockXPacket* in3 = RockXInputGetPacketAt(input, 2);

    RockXPacketSetInt(in1, 1);
    RockXPacketSetInt(in2, 2);
    RockXPacketSetInt(in3, 3);

    // 执行
    RockXProcess(handle, input, output);

    RockXPacket* packet_objects = RockXOutputGetPacketAt(output, 0);
    int v = RockXPacketGetInt(packet_objects, -1);
    printf("output: %d\n", v);

    // 销毁释放
    RockXInputDestroy(handle, input);
    RockXOutputDestroy(handle, output);

    RockXDestroy(handle);
```

调用函数示例

自定义模块

模块实现单一的算法功能，可以被单独调用，也可以组合到计算图中。

算法SDK使用RockXModule结构体类型来表示一个算法模块，其中定义了模块名称、输入输出数据、模块类型、回调函数等信息，如下代码所示

```
typedef struct RockXModule
{
    // Header info
    uint32_t magic;                  ///< Magic number
    uint32_t version;                ///< Version number
    uint32_t size;                   ///< Size of the module
    uint32_t flags;                  ///< Flags
    // Basic info
    char* name;                      ///< Module name
    int inNum;                       ///< Number of module inputs
    int outNum;                      ///< Number of module outputs
    int shareNum;                    ///< Number of module shared data
    int paramNum;                    ///< Number of module parameters
    const char** inputs;             ///< Module input names
    const char** outputs;            ///< Module output names
    const char** shares;             ///< Module shared data names
    const char** params;             ///< Module parameter names
    RockXModuleType type;            ///< Module type
    // Callback functions
    OnModuleBuilder onBuilder;       ///< Subgraph type building callback function
    OnModuleRegister onRegister;     ///< Registration callback function
    OnModuleUnRegister onUnRegister; ///< Unregistration callback function
    OnModulePreInit onPreInit;       ///< Pre-initialization callback function
    OnModuleInit onInit;             ///< Initialization callback function
    OnModulePostInit onPostInit;     ///< Post-initialization callback function
    OnModuleRelease onRelease;       ///< Deinitialization callback function
    OnModuleParamSet onParamSet;     ///< Parameter update callback function
    OnModuleParamGet onParamGet;     ///< Parameter get callback function
    OnModuleInputGet onInputsGet;    ///< Callback function to pre-get input Tensor
information and memory
    OnModuleOutputGet onOutputsGet;  ///< Callback function to pre-get output Tensor
information and memory
    OnModuleProcess onProcess;       ///< Processing callback function
} RockXModule;
```

模块类型RockXModuleType包括以下两种：

- ROCKX_MODULE_TYPE_ATOMIC： 原子模块，表示一个单一的算法功能
- ROCKX_MODULE_TYPE_SUBGRAPH： 子图模块，表示一个计算子图，子图信息通过`onBuilder`回调函数提供

模块需要定义一个按照特定规则命名的函数，才能被框架通过模块名加载。函数的命名规则为`GetRockXModule<ModuleName>`，其中`<ModuleName>`为模块名。
框架加载模块时，会根据该规则查找当前进程或加载的算法SDK的库中是否存在该函数，如果存在则调用该函数获取到对应的RockXModule的实例。

一个简单的模块定义示例如下：

```
#include <rockx.h>
#include <rockx_type.h>
#include <rockx_module.h>

#define MODULE_NAME "Add"

// 输入名称列表
static const char* INPUTS_NAME[] = {
    "in1",
    "in2"
};

// 输出名称列表
static const char* OUTPUTS_NAME[] = {
    "out"
};

// 输入输出数量
#define INPUTS_NUM (sizeof(INPUTS_NAME) / sizeof(INPUTS_NAME[0]))
#define OUTPUTS_NUM (sizeof(OUTPUTS_NAME) / sizeof(OUTPUTS_NAME[0]))

// 模块自定义上下文
typedef struct{
    int option;
} RockXAddContext;

// 模块初始化回调函数
static int Add_OnModuleInit(RockXModuleContext ctx)
{
    LOGD("Add_OnModuleInit");
    int ret;

    RockXAddContext* module_ctx = malloc(sizeof(RockXAddContext));
    if (module_ctx == NULL) {
        return -1;
    }
    // ...

    RockXModuleSetExtData(ctx, module_ctx);
    return 0;
}

// 模块释放回调函数
static int Add_OnModuleRelease(RockXModuleContext ctx)
{
    LOGD("Add_OnModuleRelease");
    if (ctx == NULL) {
        return -1;
    }
    RockXAddContext* module_ctx = RockXModuleGetExtData(ctx);
    if (module_ctx != NULL) {
        free(module_ctx);
    }
```

```
    return 0;
}

// 模块选项配置回调函数
static int Add_OnModuleParamSet(RockXModuleContext ctx, const RockXParam* param)
{
    RockXAddContext* module_ctx = RockXModuleGetExtData(ctx);

    return 0;
}

// 模块处理回调函数
static int Add_OnModuleProcess(const RockXModuleContext ctx, RockXPacket* inData, int inNum,
RockXPacket* outData,
                                    int outNum)
{
    RockXAddContext* module_ctx = RockXModuleGetExtData(ctx);

    // ...
    return 0;
}

// 模块获取接口
RockXModule* GetRockXModuleAdd()
{
    static RockXModule module = {
        .name = MODULE_NAME,
        .inNum = INPUTS_NUM,
        .outNum = OUTPUTS_NUM,
        .inputs = INPUTS_NAME,
        .outputs = OUTPUTS_NAME,
        .type = ROCKX_MODULE_TYPE_ATOMIC,
        .onInit = Add_OnModuleInit,
        .onRelease = Add_OnModuleRelease,
        .onParamSet = Add_OnModuleParamSet,
        .onProcess = Add_OnModuleProcess,
    };
    return &module;
}
```

应用自定义模块后可以通过注册接口（RockXModuleRegister/RockXModuleGlobalRegister）将模块注册到SDK中

自定义计算图

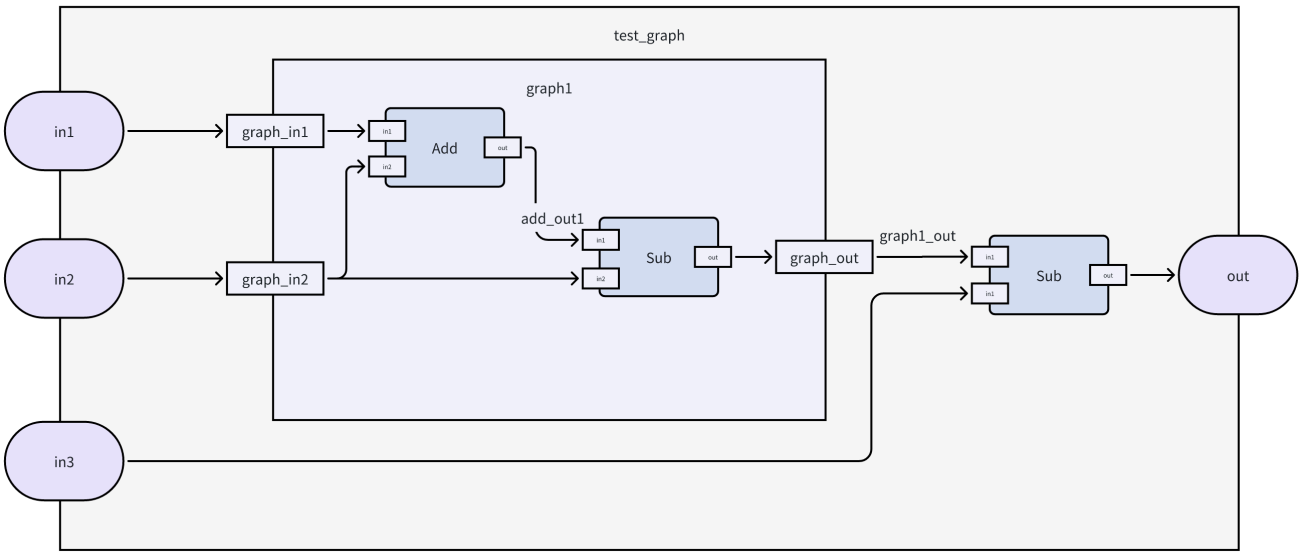
计算图可以用来表示包含多个算法模块的算法任务，例如人脸识别、车牌识别、文本转语音等，由多个模块组成。

计算图通过RockXGraphInfo结构体类型进行定义，结构体字段包括了计算图的名称、输入输出信息、配置选项信息、子节点等信息

节点通过RockXNodeInfo接口定义，包含节点的名称、输入输出、共享数据、配置、子节点等信息

计算图可以通过`rockx_builder.h`中定义的接口进行代码构建，也可以支持json进行描述（还在开发），具体接口参考计算图构建接口

示例




```
RockXGraphInfo* GetRockXGraphCalculator()
{
    // 创建计算图
    RockXGraphInfo* graph = RockXBuilderGraphCreate("test_graph");

    // 定义输入输出
    RockXBuilderGraphAddInput(graph, "in1");
    RockXBuilderGraphAddInput(graph, "in2");
    RockXBuilderGraphAddInput(graph, "in3");
    RockXBuilderGraphAddOutput(graph, "out");

    // 创建一个子图
    RockXNodeInfo* node_graph1 = RockXBuilderNodeCreate("graph1", "SubGraph");
    RockXBuilderNodeAddInput(node_graph1, "graph_in1", "in1");
    RockXBuilderNodeAddInput(node_graph1, "graph_in2", "in2");
    RockXBuilderNodeAddOutput(node_graph1, "graph_out", "graph1_out1");
    RockXBuilderAddNode(graph, node_graph1);

    // 创建一个Add节点，添加到子图
    RockXNodeInfo* node_add1 = RockXBuilderNodeCreate("add1", "Add");
    RockXBuilderNodeAddInput(node_add1, "in1", "graph_in1");
    RockXBuilderNodeAddInput(node_add1, "in2", "graph_in2");
    RockXBuilderNodeAddOutput(node_add1, "out", "add_out1");
    RockXBuilderAddNode(node_graph1, node_add1);

    // 创建一个Sub节点，添加到子图
    RockXNodeInfo* node_sub1 = RockXBuilderNodeCreate("sub1", "Sub");
    RockXBuilderNodeAddInput(node_sub1, "in1", "add_out1");
    RockXBuilderNodeAddInput(node_sub1, "in2", "graph_in2");
    RockXBuilderNodeAddOutput(node_sub1, "out", "graph_out");
    RockXBuilderAddNode(node_graph1, node_sub1);

    // 创建一个Sub，添加到计算图中
```

```
    RockXNodeInfo* node_sub2 = RockXBuilderNodeCreate("sub2", "Sub");
    RockXBuilderNodeAddInput(node_sub2, "in1", "graph1_out1");
    RockXBuilderNodeAddInput(node_sub2, "in2", "in3");
    RockXBuilderNodeAddOutput(node_sub2, "out", "out");
    RockXBuilderAddNode(graph, node_sub2);

    return graph;
}
```

API参考

主要接口

函数定义

函数	描述
RockXCreate	创建一个实例
RockXDestroy	销毁一个实例
RockXLoadSDK	加载算法SDK
RockXInit	实例初始化
RockXSetParam	设置节点参数
RockXGetParam	获取节点参数
RockXSetNodeState	设置节点状态（运行、停止、暂停、继续）
RockXSetInputInfo	添加一个输入
RockXSetOutputInfo	添加一个输出
RockXRemoveOutputInfo	移除一个输出
RockXInputCreate	创建输入
RockXInputDestroy	销毁输入
RockXOutputCreate	创建输出
RockXOutputDestroy	销毁输出
RockXProcess	处理执行
RockXProcessAsync	异步处理执行
RockXWaitFinish	等待异步执行完成
RockXModuleRegister	注册一个模块
RockXModuleUnRegister	注销一个模块
RockXModuleGlobalRegister	注册一个全局模块

函数	描述
RockXModuleGlobalUnRegister	注销一个全局模块
RockXResultCallback	异步结果回调
RockXInputPacketsNum	输入数据包数量
RockXOutputPacketsNum	输出数据包数量
RockXInputGetPacketAt	从输入获取一个数据包
RockXOutputGetPacketAt	从输出获取一个数据包
RockXInputGetPacketByName	根据包名从输入获取一个数据包
RockXOutputGetPacketByName	根据包名从输出获取一个数据包

RockXCreate

【描述】

创建一个实例

【声明】

```
RockXHandle RockXCreate(uint32_t flag, void* userdata);
```

【输入】

参数	输入/输出	类型	说明
flag	输入	uint32_t	初始化配置标志，当前支持标志如下： **ROCKX_FLAG_ASYNC_EXECUTE_MASK**：需要异步执行标志。
userdata	输入	void*	用户自定义指针，异步时回调函数返回

【返回值】

RockXHandle：实例句柄

【说明】

无

RockXDestroy

【描述】

销毁一个实例

【声明】

```
RockXRetCode RockXDestroy(RockXHandle handle);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXLoadSDK

【描述】

加载算法SDK

【声明】

```
RockXRetCode RockXLoadSDK(RockXHandle handle, const char* path);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
path	输入	const char*	算法SDK目录路径

【返回值】

RockXRetCode：返回操作状态码

【说明】

可以多次调用，加载多个算法SDK

RockXInit

【描述】

实例初始化

【声明】

```
RockXRetCode RockXInit(RockXHandle handle);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXSetParam

【描述】

设置节点参数

【声明】

```
RockXRetCode RockXSetParam(RockXHandle handle, const char* nodeName, const RockXParam* param);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
nodeName	输入	const char*	节点名称（设为NULL表示全局参数，需要按照参数配置章节设置节点路径）
param	输入	const RockXParam*	如果节点名设为NULL，表示设置一个全局参数参数

【返回值】

RockXRetCode：返回操作状态码

【说明】

RockXGetParam

【描述】

获取参数

【声明】

```
RockXParam* RockXGetParam(RockXHandle handle, const char* nodeName, const char* paramName);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
nodeName	输入	const char*	节点名称（可选，为NULL时获取全局参数）
paramName	输入	const char*	参数名称

【返回值】

RockXParam*：返回参数指针

【说明】

无

RockXSetNodeState

【描述】

更新节点状态

【声明】

```
RockXRetCode RockXSetNodeState(RockXHandle handle, const char* nodeName, RockXNodeAction action);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
nodeName	输入	const char*	节点名称
action	输入	RockXNodeAction	节点动作

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXSetInputInfo

【描述】

配置需要的输入，RockXInputCreate将根据配置创建输入

【声明】

```
RockXRetCode RockXSetInputInfo(RockXHandle handle, const char* name, const char* inNames[],
int inNum);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
name	输入	const char*	输入名称
inNames	输入	const char*[]	输入名称列表
inNum	输入	int	输入数量

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXSetOutputInfo

【描述】

配置需要的输出，对于两种执行方式：

- 同步模式时，callback可以置空，后面调用RockXOutputCreate将根据配置创建输出
- 异步模式时，可以设置callback回调函数和回调模式

【声明】

```
RockXRetCode RockXSetOutputInfo(RockXHandle handle, const char* name, const char*
outNames[], int outNum, RockXResultCallback callback, RockXCallbackMode mode);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
name	输入	const char*	输入名称
outNames	输入	const char*[]	输出名称列表
outNum	输入	int	输出数量
callback	输入	RockXResultCallback	回调函数
mode	输入	RockXCallbackMode	回调模式

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXRemoveOutputInfo

【描述】

移除输出配置信息

【声明】

```
RockXRetCode RockXRemoveOutputInfo(RockXHandle handle, const char* name);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
name	输入	const char*	输入名称

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXInputCreate

【描述】

创建输入

【声明】

```
RockXInput* RockXInputCreate(RockXHandle handle);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄

【返回值】

RockXInput*：返回输入指针

【说明】

无

RockXInputDestroy

【描述】

释放输入

【声明】

```
RockXRetCode RockXInputDestroy(RockXHandle handle, RockXInput* input);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
input	输入	RockXInput*	输入指针

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXOutputCreate

【描述】

创建输出

【声明】

```
RockXOutput* RockXOutputCreate(RockXHandle handle, const char* outNames[], int outNum);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
outNames	输入	const char*[]	输出名称列表
outNum	输入	int	输出数量

【返回值】

RockXOutput*：返回输出指针

【说明】

无

RockXOutputDestroy

【描述】

释放输出

【声明】

```
RockXRetCode RockXOutputDestroy(RockXHandle handle, RockXOutput* output);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
output	输入	RockXOutput*	输出指针

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXProcess

【描述】

处理执行

【声明】

```
RockXRetCode RockXProcess(RockXHandle handle, RockXInput* input, RockXOutput* output);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
input	输入	RockXInput*	输入数据
output	输出	RockXOutput*	输出数据

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXProcessAsync

【描述】

异步处理执行

【声明】

```
RockXRetCode RockXProcessAsync(RockXHandle handle, RockXInput* input, void** wait);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
input	输入	RockXInput*	输入
wait	输出	void**	等待句柄

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXWaitFinish

【描述】

等待异步执行完成

【声明】

```
RockXRetCode RockXWaitFinish(RockXHandle handle, void* wait);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
wait	输入	void*	等待句柄

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXInputPacketsNum

【描述】

获取输入数据包数量

【声明】

```
int RockXInputPacketsNum(RockXInput* input);
```

【输入】

参数	输入/输出	类型	说明
input	输入	RockXInput*	输入指针

【返回值】

int：返回数据包数量

【说明】

无

RockXOutputPacketsNum

【描述】

获取输出数据包数量

【声明】

```
int RockXOutputPacketsNum(RockXOutput* output);
```

【输入】

参数	输入/输出	类型	说明
output	输入	RockXOutput*	输出指针

【返回值】

int：返回数据包数量

【说明】

无

RockXInputGetPacketAt

【描述】

获取输入数据包

【声明】

```
RockXPacket* RockXInputGetPacketAt(RockXInput* input, int index);
```

【输入】

参数	输入/输出	类型	说明
input	输入	RockXInput*	输入指针
index	输入	int	索引

【返回值】

RockXPacket*：返回数据包指针

【说明】

无

RockXOutputGetPacketAt

【描述】

获取输出数据包

【声明】

```
RockXPacket* RockXOutputGetPacketAt(RockXOutput* output, int index);
```

【输入】

参数	输入/输出	类型	说明
output	输入	RockXOutput*	输出指针
index	输入	int	索引

【返回值】

RockXPacket*：返回数据包指针

【说明】

无

RockXInputGetPacketByName

【描述】

获取输入数据包

【声明】

```
RockXPacket* RockXInputGetPacketByName(RockXInput* input, const char* name);
```

【输入】

参数	输入/输出	类型	说明
input	输入	RockXInput*	输入指针
name	输入	const char*	包名称

【返回值】

RockXPacket*：返回数据包指针

【说明】

无

RockXOutputGetPacketByName

【描述】

获取输出数据包

【声明】

```
RockXPacket* RockXOutputGetPacketByName(RockXOutput* output, const char* name);
```

【输入】

参数	输入/输出	类型	说明
output	输入	RockXOutput*	输出指针
name	输入	const char*	包名称

【返回值】

RockXPacket*：返回数据包指针

【说明】

无

结构体定义

结构体	描述
RockXInput	输入数据
RockXOutput	输出数据
RockXPacket	数据包
RockXPacketCallback	数据包回调
RockXParam	参数
RockXParamCallback	参数回调
RockXValue	数值
RockXTensor	张量
RockXTensorShape	张量形状
RockXTensorQntInfo	张量量化信息
RockXImage	图像
RockXAudioWav	音频数据
RockXMemoryBuffer	缓冲区内存

RockXInput

【描述】

输入数据

【声明】

```
typedef struct RockXInput RockXInput;
```

【字段】

该结构体为不透明类型（可以通过RockXInputXXX函数进行操作）

【说明】

无

RockXOutput

【描述】

输出数据

【声明】

```
typedef struct RockXOutput RockXOutput;
```

【字段】

该结构体为不透明类型（可以通过RockXOutputXXX函数进行操作）

【说明】

无

RockXPacket

【描述】

数据包

【声明】

```
typedef struct RockXPacket RockXPacket;
```

【字段】

该结构体为不透明类型（可以通过RockXPacketGetXXX或RockXPacketSetXXX函数操作）

【说明】

无

RockXParam

【描述】

参数

【声明】

```
typedef struct RockXParam RockXParam;
```

【字段】

该结构体为不透明类型（可以通过RockXParamGetXXX或RockXParamSetXXX函数操作）

【说明】

无

RockXParamCallback

【描述】

参数回调

【声明】

```
typedef struct RockXParamCallback
{
    RockXParamReleaseCallback OnRelease;
    RockXParamCopyCallback OnCopy;
    RockXParamDumpCallback OnDump;
} RockXParamCallback;
```

【字段】

字段	类型	说明
OnRelease	RockXParamReleaseCallback	释放回调
OnCopy	RockXParamCopyCallback	复制回调
OnDump	RockXParamDumpCallback	导出回调

【说明】

无

RockXValue

【描述】

数值

【声明】

```
typedef struct RockXValue
{
    RockXValueType type;
    RockXDataType dtype;
    union
    {
        void* p;
        char* s;
        int i;
        long l;
        float f;
        double d;
    };
    size_t count;
    size_t size;
    uint8_t needRelease;
} RockXValue;
```

【字段】

字段	类型	说明
type	RockXValueType	值类型

字段	类型	说明
dtype	RockXDataType	数据类型
p	void*	指针类型值
s	char*	字符串类型值
i	int	整型值
l	long	长整型值
f	float	浮点型值
d	double	双精度浮点型值
count	size_t	元素数量
size	size_t	内存大小
needRelease	uint8_t	是否需要释放内存

【说明】

无

RockXMemoryBuffer

【描述】

缓冲区内存

【声明】

```
typedef struct RockXMemoryBuffer
{
    RockXMemType type;
    size_t size;
    void* virtAddr;
    void* phyAddr;
    int fd;
    int handle;
    void* extdata;
    const char* tag;
} RockXMemoryBuffer;
```

【字段】

字段	类型	说明
type	RockXMemType	内存类型
size	size_t	内存大小

字段	类型	说明
virtAddr	void*	虚拟地址
phyAddr	void*	物理地址
fd	int	文件描述符
handle	int	句柄
extdata	void*	扩展数据
tag	const char*	标签

【说明】

无

RockXTensorQntInfo

【描述】

张量量化信息

【声明】

```
typedef struct
{
    RockXTensorQntType qntType;
    int8_t fl;
    int32_t zp;
    float scale;
} RockXTensorQntInfo;
```

【字段】

字段	类型	说明
qntType	RockXTensorQntType	量化类型
fl	int8_t	DFP量化的小数位数
zp	int32_t	非对称量化的零点值
scale	float	非对称量化的缩放因子

【说明】

无

RockXTensorShape

【描述】

张量形状

【声明】

```
typedef struct {
    int dims[16];
    int ndims;
    RockXTensorFormat format;
} RockXTensorShape;
```

【字段】

字段	类型	说明
dims	int[16]	维度数组
ndims	int	维度数量
format	RockXTensorFormat	数据排列格式

【说明】

无

RockXTensor

【描述】

张量

【声明】

```
typedef struct RockXTensor {
    RockXTensorShape shape;
    size_t nelems;
    RockXDataType dtype;
    RockXMemoryBuffer buffer;
    RockXTensorQntInfo qntInfo;
} RockXTensor;
```

【字段】

字段	类型	说明
shape	RockXTensorShape	张量形状
nelems	size_t	元素数量
dtype	RockXDataType	数据类型

字段	类型	说明
buffer	RockXMemoryBuffer	内存缓冲区
qntInfo	RockXTensorQntInfo	量化信息

【说明】

无

RockXImage

【描述】

图像

【声明】

```
typedef struct RockXImage
{
    int width;
    int height;
    RockXPixelFormat format;
    RockXMemoryBuffer buffer;
} RockXImage;
```

【字段】

字段	类型	说明
width	int	图像宽度
height	int	图像高度
format	RockXPixelFormat	像素格式
buffer	RockXMemoryBuffer	内存缓冲区

【说明】

无

RockXAudioWav

【描述】

音频数据

【声明】

```
typedef struct RockXAudioWav {
    int sampleRate;
    int channels;
    RockXStreamState state;
    unsigned long timestamp;
    int sampleNum;
    RockXDataType dtype;
    void* data;
} RockXAudioWav;
```

【字段】

字段	类型	说明
sampleRate	int	采样率
channels	int	通道数
state	RockXStreamState	流状态
timestamp	unsigned long	时间戳
sampleNum	int	采样点数
dtype	RockXDataType	数据类型
data	void*	音频数据

【说明】

无

RockXKeyValue

【描述】

键值对

【声明】

```
typedef struct RockXKeyValue
{
    char* key;
    char* value;
} RockXKeyValue;
```

【字段】

字段	类型	说明
key	char*	键
value	char*	值

【说明】

无

枚举定义

枚举	描述
RockXRetCode	返回错误码
RockXPacketState	数据包状态
RockXPacketMode	数据包模式
RockXPacketType	数据包类型
RockXDataType	数据类型
RockXNodeAction	节点操作
RockXModuleInstanceMode	模块实例模式
RockXCallbackMode	回调模式
RockXStreamState	流状态
RockXMemType	内存类型
RockXTensorFormat	张量数据排列格式
RockXTensorQntType	张量量化类型
RockXPixelFormat	图像像素格式
RockXImageTransform	图像旋转模式
RockXValueType	值（RockXValue）类型

RockXRetCode

【描述】

返回错误码

【枚举值】

枚举	说明
ROCKX_RET_SUCCESS	成功（0）
ROCKX_RET_FAIL	失败（-1）
ROCKX_RET_PARAM_ERROR	参数错误（-2）
ROCKX_RET_INIT_ERROR	初始化错误（-3）

枚举	说明
ROCKX_RET_MEM_ERROR	内存错误（-4）
ROCKX_RET_AUTH_ERROR	授权错误（-99）

RockXPacketState

【描述】

数据包状态

【枚举值】

枚举	说明
ROCKX_PACKET_STATE_NONE	初始状态
ROCKX_PACKET_STATE_DOING	数据包正在处理中
ROCKX_PACKET_STATE_READY	数据包处理完成
ROCKX_PACKET_STATE_OUTPUT	数据包已输出
ROCKX_PACKET_STATE_RELEASE	数据包已释放
ROCKX_PACKET_STATE_ERROR	数据包处理错误
ROCKX_PACKET_STATE_INVALID	数据包无效

RockXPacketMode

【描述】

数据包模式

【枚举值】

枚举	说明
ROCKX_PACKET_MODE_COPY	默认模式，复制数据
ROCKX_PACKET_MODE_SHARED	共享模式，支持多个数据包共享数据，使用RockXPacketShareData进行共享数据管理
ROCKX_PACKET_MODE_UNIQUE	独占模式，只有一个数据包可以独占管理数据，使用RockXPacketMoveData进行数据管理转移

RockXPacketType

【描述】

数据包类型

【枚举值】

枚举	说明
ROCKX_PACKET_TYPE_NONE	无类型
ROCKX_PACKET_TYPE_IMAGE	图像类型
ROCKX_PACKET_TYPE_AUDIO	音频类型
ROCKX_PACKET_TYPE_TENSOR	张量类型
ROCKX_PACKET_TYPE_VALUE	值类型

RockXDataType

【描述】

数据类型

【枚举值】

枚举	说明
ROCKX_DATA_TYPE_UINT8	无符号8位整数
ROCKX_DATA_TYPE_INT8	有符号8位整数
ROCKX_DATA_TYPE_UINT16	无符号16位整数
ROCKX_DATA_TYPE_INT16	有符号16位整数
ROCKX_DATA_TYPE_UINT32	无符号32位整数
ROCKX_DATA_TYPE_INT32	有符号32位整数
ROCKX_DATA_TYPE_UINT64	无符号64位整数
ROCKX_DATA_TYPE_INT64	有符号64位整数
ROCKX_DATA_TYPE_FLOAT	32位浮点数
ROCKX_DATA_TYPE_DOUBLE	64位浮点数
ROCKX_DATA_TYPE_FLOAT16	16位浮点数
ROCKX_DATA_TYPE_RAW	原始数据
ROCKX_DATA_TYPE_STRUCT	结构体数据

枚举	说明
ROCKX_DATA_TYPE_MAX	最大值

RockXNodeAction

【描述】

节点操作

【枚举值】

枚举	说明
ROCKX_NODE_ACTION_START	启动节点
ROCKX_NODE_ACTION_STOP	停止节点
ROCKX_NODE_ACTION_PAUSE	暂停节点
ROCKX_NODE_ACTION_RESUME	恢复节点
ROCKX_NODE_ACTION_RESET	重置节点
ROCKX_NODE_ACTION_PASS	跳过节点

RockXModuleInstanceMode

【描述】

模块实例模式

【枚举值】

枚举	说明
ROCKX_MODULE_INSTANCE_MODE_DEFAULT	默认模式，每次创建新的模块实例
ROCKX_MODULE_INSTANCE_MODE_REUSE	复用模式，每次复用已存在的模块实例

RockXCallbackMode

【描述】

回调模式

【枚举值】

枚举	说明
ROCKX_CALLBACK_NONE	不需要回调
ROCKX_CALLBACK_ON_SCHED_THREAD	在调度线程中异步回调结果，会阻塞当前帧

枚举	说明
ROCKX_CALLBACK_ON_SEPARATE_THREAD	在独立线程中异步回调结果

RockXStreamState

【描述】

流状态

【枚举值】

枚举	说明
ROCKX_STREAM_STATE_FIRST	流的第一个数据包
ROCKX_STREAM_STATE_CONTINUE	流的中间连续数据包
ROCKX_STREAM_STATE_LAST	流的最后一个数据包

RockXMemType

【描述】

内存类型

【枚举值】

枚举	说明
ROCKX_MEM_TYPE_UNKNOW	未知类型
ROCKX_MEM_TYPE_CPU	CPU内存
ROCKX_MEM_TYPE_DMA_BUF	DMA缓冲区
ROCKX_MEM_TYPE_DMA_HEAP	RK DMA堆

RockXTensorFormat

【描述】

张量数据排列格式

【枚举值】

枚举	说明
ROCKX_TENSOR_FORMAT_NCHW	NCHW格式
ROCKX_TENSOR_FORMAT_NHWC	NHWC格式
ROCKX_TENSOR_FORMAT_NC1HWC2	NC1HWC2格式

枚举	说明
ROCKX_TENSOR_FORMAT_MAX	最大值

RockXTensorQntType

【描述】

张量量化类型

【枚举值】

枚举	说明
ROCKX_TENSOR_QNT_NONE	无量化
ROCKX_TENSOR_QNT_DFP	动态定点量化
ROCKX_TENSOR_QNT_AFFINE_ASYMMETRIC	非对称仿射量化
ROCKX_TENSOR_QNT_MAX	最大值

RockXPixelFormat

【描述】

图像像素格式

【枚举值】

枚举	说明
ROCKX_PIXEL_FORMAT_UNKNOW	未知二进制数据
ROCKX_PIXEL_FORMAT_GRAY8	8位灰度图
ROCKX_PIXEL_FORMAT_GRAY16	16位灰度图
ROCKX_PIXEL_FORMAT_RGB888	RGB888格式
ROCKX_PIXEL_FORMAT_BGR888	BGR888格式
ROCKX_PIXEL_FORMAT_RGBA8888	RGBA8888格式
ROCKX_PIXEL_FORMAT_BGRA8888	BGRA8888格式
ROCKX_PIXEL_FORMAT_YUV420P_YU12	YUV420P YU12格式
ROCKX_PIXEL_FORMAT_YUV420P_YV12	YUV420P YV12格式
ROCKX_PIXEL_FORMAT_YUV420SP_NV12	YUV420SP NV12格式
ROCKX_PIXEL_FORMAT_YUV420SP_NV21	YUV420SP NV21格式

枚举	说明
ROCKX_PIXEL_FORMAT_YUV422P_YU16	YUV422P YU16格式
ROCKX_PIXEL_FORMAT_YUV422P_YV16	YUV422P YV16格式
ROCKX_PIXEL_FORMAT_YUV422SP_NV16	YUV422SP NV16格式
ROCKX_PIXEL_FORMAT_YUV422SP_NV61	YUV422SP NV61格式
ROCKX_PIXEL_FORMAT_YUV422_YUYV	YUV422 YUYV格式
ROCKX_PIXEL_FORMAT_YUV422_YVYU	YUV422 YVYU格式
ROCKX_PIXEL_FORMAT_JPEG	JPEG格式
ROCKX_PIXEL_FORMAT_MAX	最大值

RockXImageTransform

【描述】

图像旋转模式

【枚举值】

枚举	说明
ROCKX_IMAGE_TRANSFORM_NONE	正常
ROCKX_IMAGE_TRANSFORM_FLIP_H	水平翻转
ROCKX_IMAGE_TRANSFORM_FLIP_V	垂直翻转
ROCKX_IMAGE_TRANSFORM_ROTATE_90	顺时针旋转90度
ROCKX_IMAGE_TRANSFORM_ROTATE_180	顺时针旋转180度
ROCKX_IMAGE_TRANSFORM_ROTATE_270	顺时针旋转270度
ROCKX_IMAGE_TRANSFORM_TRANSPOSE	转置

RockXValueType

【描述】

值（RockXValue）类型

【枚举值】

枚举	说明
ROCKX_VALUE_TYPE_NONE	无类型

枚举	说明
ROCKX_VALUE_TYPE_NUMBER	数值类型
ROCKX_VALUE_TYPE_STRING	字符串类型
ROCKX_VALUE_TYPE_ARRAY	数组类型
ROCKX_VALUE_TYPE_POINTER	指针类型

模块接口

函数定义

函数接口	描述
RockXModuleRegister	注册一个模块
RockXModuleUnRegister	注销一个模块
RockXModuleGlobalRegister	注册一个全局模块
RockXModuleGlobalUnRegister	注销一个全局模块
RockXModuleSetExtData	设置模块上下文自定义数据
RockXModuleGetExtData	获取模块上下文自定义数据
RockXModuleGetSharedDataAt	获取共享数据包
RockXModuleGetModule	获取模块对象
RockXModuleGetParam	获取配置参数
RockXModuleGetParamInt	获取整型配置参数
RockXModuleGetParamFloat	获取浮点型配置参数
RockXModuleGetParamString	获取字符串型配置参数
RockXModuleGetParamPointer	获取指针型配置参数
RockXModuleGetParamAt	获取模块自身参数
RockXModuleGetParamIntAt	获取模块自身整型参数
RockXModuleGetParamFloatAt	获取模块自身浮点型参数
RockXModuleGetParamStringAt	获取模块自身字符串参数
RockXModuleGetParamPointerAt	获取模块自身指针参数
RockXModuleSetParam	设置参数
RockXModuleAuthCheck	模块授权检查

RockXModuleRegister

【描述】

注册一个模块

【声明】

```
RockXRetCode RockXModuleRegister(RockXHandle handle, RockXModule* module);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
module	输入	RockXModule*	模块对象

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXModuleUnRegister

【描述】

注销一个模块

【声明】

```
RockXRetCode RockXModuleUnRegister(RockXHandle handle, const char* moduleName);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
moduleName	输入	const char*	模块名

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXModuleGlobalRegister

【描述】

注册一个全局模块

【声明】

```
RockXRetCode RockXModuleGlobalRegister(RockXModule* module);
```

【输入】

参数	输入/输出	类型	说明
module	输入	RockXModule*	模块对象

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXModuleGlobalUnRegister

【描述】

注销一个全局模块

【声明】

```
RockXRetCode RockXModuleGlobalUnRegister(const char* moduleName);
```

【输入】

参数	输入/输出	类型	说明
moduleName	输入	const char*	模块名

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXModuleSetExtData

【描述】

设置模块上下文自定义数据

【声明】

```
int RockXModuleSetExtData(RockXModuleContext context, void* extData);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文
extData	输入	void*	自定义数据

【返回值】

int：0表示成功，其他为失败

【说明】

无

RockXModuleGetExtData

【描述】

获取模块上下文自定义数据

【声明】

```
void* RockXModuleGetExtData(RockXModuleContext context);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文

【返回值】

void*：自定义数据指针

【说明】

无

RockXModuleGetSharedDataAt

【描述】

获取共享数据包

【声明】

```
RockXPacket* RockXModuleGetSharedDataAt(RockXModuleContext context, int index);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文
index	输入	int	共享数据索引

【返回值】

RockXPacket*：共享数据包指针

【说明】

无

RockXModuleGetModule

【描述】

获取模块对象

【声明】

```
RockXModule* RockXModuleGetModule(RockXModuleContext context);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文

【返回值】

RockXModule*：模块对象指针

【说明】

无

RockXModuleGetParam

【描述】

获取配置参数

【声明】

```
RockXParam* RockXModuleGetParam(RockXModuleContext context, RockXParamScope scope, const char* nodeName, const char* name);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文
scope	输入	RockXParamScope	参数作用域
nodeName	输入	const char*	节点名称
name	输入	const char*	参数名称

【返回值】

RockXParam*：参数指针

【说明】

无

RockXModuleGetParamInt

【描述】

获取整型配置参数

【声明】

```
int RockXModuleGetParamInt(RockXModuleContext context, RockXParamScope scope, const char* nodeName, const char* name, int defval);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文
scope	输入	RockXParamScope	参数作用域
nodeName	输入	const char*	节点名称
name	输入	const char*	参数名称

参数	输入/输出	类型	说明
defval	输入	int	默认值

【返回值】

int：参数值

【说明】

无

RockXModuleGetParamFloat

【描述】

获取浮点型配置参数

【声明】

```
float RockXModuleGetParamFloat(RockXModuleContext context, RockXParamScope scope, const char* nodeName, const char* name, float defval);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文
scope	输入	RockXParamScope	参数作用域
nodeName	输入	const char*	节点名称
name	输入	const char*	参数名称
defval	输入	float	默认值

【返回值】

float：参数值

【说明】

无

RockXModuleGetParamString

【描述】

获取字符串型配置参数

【声明】

```
char* RockXModuleGetParamString(RockXModuleContext context, RockXParamScope scope, const
char* nodeName, const char* name, const char* defval);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文
scope	输入	RockXParamScope	参数作用域
nodeName	输入	const char*	节点名称
name	输入	const char*	参数名称
defval	输入	const char*	默认值

【返回值】

char*：参数值

【说明】

无

RockXModuleGetParamPointer

【描述】

获取指针型配置参数

【声明】

```
void* RockXModuleGetParamPointer(RockXModuleContext context, RockXParamScope scope, const
char* nodeName, const char* name, const void* defval);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文
scope	输入	RockXParamScope	参数作用域
nodeName	输入	const char*	节点名称
name	输入	const char*	参数名称
defval	输入	const void*	默认值

【返回值】

void*：参数值

【说明】

无

RockXModuleGetParamAt

【描述】

获取模块自身参数

【声明】

```
RockXParam* RockXModuleGetParamAt(RockXModuleContext context, int index);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文
index	输入	int	参数索引

【返回值】

RockXParam*：参数指针

【说明】

无

RockXModuleGetParamIntAt

【描述】

获取模块自身整型参数

【声明】

```
int RockXModuleGetParamIntAt(RockXModuleContext context, int index, int defval);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文
index	输入	int	参数索引
defval	输入	int	默认值

【返回值】

int：参数值

【说明】

无

RockXModuleGetParamFloatAt

【描述】

获取模块自身浮点型参数

【声明】

```
float RockXModuleGetParamFloatAt(RockXModuleContext context, int index, float defval);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文
index	输入	int	参数索引
defval	输入	float	默认值

【返回值】

float：参数值

【说明】

无

RockXModuleGetParamStringAt

【描述】

获取模块自身字符串参数

【声明】

```
char* RockXModuleGetParamStringAt(RockXModuleContext context, int index, const char*
defval);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文
index	输入	int	参数索引
defval	输入	const char*	默认值

【返回值】

char*：参数值

【说明】

无

RockXModuleGetParamPointerAt

【描述】

获取模块自身指针参数

【声明】

```
void* RockXModuleGetParamPointerAt(RockXModuleContext context, int index, const void*
defval);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文
index	输入	int	参数索引
defval	输入	const void*	默认值

【返回值】

void*：参数值

【说明】

无

RockXModuleSetParam

【描述】

设置参数

【声明】

```
int RockXModuleSetParam(RockXModuleContext context, RockXParamScope scope, const char*
nodeName, const RockXParam* param);
```

【输入】

参数	输入/输出	类型	说明
context	输入	RockXModuleContext	模块上下文

参数	输入/输出	类型	说明
scope	输入	RockXParamScope	参数作用域
nodeName	输入	const char*	节点名称
param	输入	const RockXParam*	参数

【返回值】

int：0表示成功，其他为失败

【说明】

无

RockXModuleAuthCheck

【描述】

模块授权检查

【声明】

```
RockXRetCode RockXModuleAuthCheck(RockXModuleContext ctx, const char* authModule);
```

【输入】

参数	输入/输出	类型	说明
ctx	输入	RockXModuleContext	模块上下文
authModule	输入	const char*	授权模块名

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

回调函数定义

函数接口	描述
OnModuleRegister	模块注册回调类型
OnModuleUnRegister	模块注销回调类型
OnModuleBuilder	子图类型构建回调类型
OnModulePreInit	预初始化回调类型

函数接口	描述
OnModuleInit	初始化回调类型
OnModulePostInit	后初始化回调类型
OnModuleRelease	反初始化回调类型
OnModuleParamSet	参数更新回调类型
OnModuleParamGet	参数获取回调类型
OnModuleInputGet	输入信息获取回调类型
OnModuleOutputGet	输出信息获取回调类型
OnModulePreProcess	预处理回调类型
OnModuleProcess	处理回调类型
OnModulePostProcess	后处理回调类型

OnModuleRegister

【描述】

模块注册回调类型

【声明】

```
typedef int (*OnModuleRegister)(struct RockXModule* module);
```

【输入】

参数	输入/输出	类型	说明
module	输入	RockXModule*	模块对象

【返回值】

int：0表示成功，其他为失败

【说明】

无

OnModuleUnRegister

【描述】

模块注销回调类型

【声明】

```
typedef int (*OnModuleUnRegister)(struct RockXModule* module);
```

【输入】

参数	输入/输出	类型	说明
module	输入	RockXModule*	模块对象

【返回值】

int：0表示成功，<0失败

【说明】

无

OnModuleBuilder

【描述】

子图类型构建回调类型

【声明】

```
typedef int (*OnModuleBuilder)(RockXModuleContext ctx, RockXNodeInfo* nodeInfo);
```

【输入】

参数	输入/输出	类型	说明
ctx	输入	RockXModuleContext	模块上下文
nodeInfo	输入	RockXNodeInfo*	节点信息

【返回值】

int：0表示成功，<0失败

【说明】

无

OnModulePreInit

【描述】

预初始化回调类型

【声明】

```
typedef int (*OnModulePreInit)(RockXModuleContext ctx);
```

【输入】

参数	输入/输出	类型	说明
ctx	输入	RockXModuleContext	模块上下文

【返回值】

int：0表示成功，<0失败

【说明】

无

OnModuleInit

【描述】

初始化回调类型

【声明】

```
typedef int (*OnModuleInit)(RockXModuleContext ctx);
```

【输入】

参数	输入/输出	类型	说明
ctx	输入	RockXModuleContext	模块上下文

【返回值】

int：0表示成功，<0失败

【说明】

无

OnModulePostInit

【描述】

后初始化回调类型

【声明】

```
typedef int (*OnModulePostInit)(RockXModuleContext ctx);
```

【输入】

参数	输入/输出	类型	说明
ctx	输入	RockXModuleContext	模块上下文

【返回值】

int：0表示成功，<0失败

【说明】

无

OnModuleRelease

【描述】

反初始化回调类型

【声明】

```
typedef int (*OnModuleRelease)(RockXModuleContext ctx);
```

【输入】

参数	输入/输出	类型	说明
ctx	输入	RockXModuleContext	模块上下文

【返回值】

int：0表示成功，<0失败

【说明】

无

OnModuleParamSet

【描述】

参数更新回调类型

【声明】

```
typedef int (*OnModuleParamSet)(RockXModuleContext ctx, const RockXParam* param);
```

【输入】

参数	输入/输出	类型	说明
ctx	输入	RockXModuleContext	模块上下文
param	输入	const RockXParam*	参数

【返回值】

int：0表示成功，<0失败

【说明】

无

OnModuleParamGet

【描述】

参数获取回调类型

【声明】

```
typedef int (*OnModuleParamGet)(RockXModuleContext ctx, RockXParam* param);
```

【输入】

参数	输入/输出	类型	说明
ctx	输入	RockXModuleContext	模块上下文
param	输入	RockXParam*	参数

【返回值】

int：0表示成功，<0失败

【说明】

无

OnModuleInputGet

【描述】

输入信息获取回调类型

【声明】

```
typedef int (*OnModuleInputGet)(RockXModuleContext ctx, RockXPacket* input, int index);
```

【输入】

参数	输入/输出	类型	说明
ctx	输入	RockXModuleContext	模块上下文
input	输入	RockXPacket*	输入数据包
index	输入	int	输入索引

【返回值】

int：0表示成功，<0失败

【说明】

无

OnModuleOutputGet

【描述】

输出信息获取回调类型

【声明】

```
typedef int (*OnModuleOutputGet)(RockXModuleContext ctx, RockXPacket* output, int index);
```

【输入】

参数	输入/输出	类型	说明
ctx	输入	RockXModuleContext	模块上下文
output	输入	RockXPacket*	输出数据包
index	输入	int	输出索引

【返回值】

int：0表示成功，<0失败

【说明】

无

OnModulePreProcess

【描述】

预处理回调类型

【声明】

```
typedef int (*OnModulePreProcess)(RockXModuleContext ctx, RockXInput* input, RockXOutput*
output);
```

【输入】

参数	输入/输出	类型	说明
ctx	输入	RockXModuleContext	模块上下文
input	输入	RockXInput*	输入数据
output	输入	RockXOutput*	输出数据

【返回值】

int：0表示成功，<0失败

【说明】

无

OnModuleProcess

【描述】

处理回调类型

【声明】

```
typedef int (*OnModuleProcess)(RockXModuleContext ctx, RockXInput* input, RockXOutput*
output);
```

【输入】

参数	输入/输出	类型	说明
ctx	输入	RockXModuleContext	模块上下文
input	输入	RockXInput*	输入数据
output	输入	RockXOutput*	输出数据

【返回值】

int：0表示成功，<0失败

【说明】

无

OnModulePostProcess

【描述】

后处理回调类型

【声明】

```
typedef int (*OnModulePostProcess)(RockXModuleContext ctx, RockXInput* input, RockXOutput*
output);
```

【输入】

参数	输入/输出	类型	说明
ctx	输入	RockXModuleContext	模块上下文
input	输入	RockXInput*	输入数据

参数	输入/输出	类型	说明
output	输入	RockXOutput*	输出数据

【返回值】

int：0表示成功，<0失败

【说明】

无

结构体定义

结构体	描述
RockXModule	算法模块结构体
RockXModuleContext	模块上下文

RockXModule

【描述】

算法模块结构体

【声明】

```
typedef struct RockXModule
{
    // Header info
    uint32_t magic;
    uint32_t version;
    uint32_t size;
    uint32_t flags;
    // Basic info
    char* name;
    int inNum;
    int outNum;
    int shareNum;
    int paramNum;
    const char** inputs;
    const char** outputs;
    const char** shares;
    const char** params;
    RockXModuleType type;
    // Callback functions
    OnModuleBuilder onBuilder;
    OnModuleRegister onRegister;
    OnModuleUnRegister onUnRegister;
    OnModulePreInit onPreInit;
    OnModuleInit onInit;
```

```
    OnModulePostInit onPostInit;
    OnModuleRelease onRelease;
    OnModuleParamSet onParamSet;
    OnModuleParamGet onParamGet;
    OnModuleInputGet onInputsGet;
    OnModuleOutputGet onOutputsGet;
    OnModuleProcess onProcess;
} RockXModule;
```

【字段】

字段	类型	说明
magic	uint32_t	幻数
version	uint32_t	结构体版本
size	uint32_t	结构体大小
flags	uint32_t	扩展标志
name	char*	模块名
inNum	int	输入个数
outNum	int	输出个数
shareNum	int	共享数据个数
paramNum	int	参数个数
inputs	const char**	输入名称列表
outputs	const char**	输出名称列表
shares	const char**	共享数据名称列表
params	const char**	参数名称列表
type	RockXModuleType	模块类型
onBuilder	OnModuleBuilder	子图类型构建回调
onRegister	OnModuleRegister	注册回调
onUnRegister	OnModuleUnRegister	注销回调
onPreInit	OnModulePreInit	预初始化回调
onInit	OnModuleInit	初始化回调
onPostInit	OnModulePostInit	后初始化回调
onRelease	OnModuleRelease	反初始化回调
onParamSet	OnModuleParamSet	参数更新回调

字段	类型	说明
onParamGet	OnModuleParamGet	参数获取回调
onInputsGet	OnModuleInputGet	输入信息获取回调
onOutputsGet	OnModuleOutputGet	输出信息获取回调
onProcess	OnModuleProcess	处理回调

【说明】

无

RockXModuleContext

【描述】

模块上下文

【声明】

```
typedef void* RockXModuleContext;
```

【字段】

字段	类型	说明
无	void*	模块上下文指针

【说明】

无

枚举定义

枚举	描述
RockXModuleType	模块类型
RockXParamScope	参数作用域

RockXModuleType

【描述】

模块类型

【枚举值】

枚举	说明
ROCKX_MODULE_TYPE_ATOMIC	原子模块
ROCKX_MODULE_TYPE_SUBGRAPH	子图模块

RockXParamScope

【描述】

参数作用域

【枚举值】

枚举	说明
ROCKX_PARAM_SCOPE_GLOBAL	全局参数
ROCKX_PARAM_SCOPE_GRAPH	所在计算图参数
ROCKX_PARAM_SCOPE_NODE	节点自身参数

好的，我将根据`rockx_builder.h`头文件内容，按照相同的格式补充完善计算图构建接口章节。

计算图构建接口

函数定义

函数接口	描述
RockXBuilderGraphCreate	创建一个计算图
RockXBuilderGraphDestroy	销毁一个计算图
RockXBuilderNodeCreate	创建一个节点
RockXBuilderNodeDestroy	销毁一个节点
RockXBuilderAddNode	添加一个节点
RockXBuilderRemoveNode	删除一个节点
RockXBuilderGraphAddInput	计算图添加一个输入
RockXBuilderGraphAddOutput	计算图添加一个输出
RockXBuilderNodeAddInput	节点添加一个输入
RockXBuilderNodeGetInput	节点输入获取
RockXBuilderNodeAddOutput	节点添加一个输出

函数接口	描述
RockXBuilderNodeGetOutput	节点输出获取
RockXBuilderNodeAddShare	节点添加一个共享数据
RockXBuilderNodeGetShare	节点共享数据获取
RockXBuilderAddParam	添加一个配置
RockXBuilderAddParamInt	添加一个整型配置
RockXBuilderAddParamFloat	添加一个浮点类型配置
RockXBuilderAddParamString	添加一个字符串类型配置
RockXBuilderAddParamPointer	添加一个指针类型配置
RockXBuilderCheckParamExist	检查配置是否存在
RockXBuilderGetParam	获取一个配置
RockXBuilderGetParamInt	获取一个整型配置
RockXBuilderGetParamString	获取一个字符串类型配置
RockXBuilderGetParamPointer	获取一个指针类型配置
RockXBuilderGetParamFloat	获取一个浮点类型配置
RockXBuilderGraphInfoMoveToNodeInfo	计算图信息转换为节点信息

RockXBuilderGraphCreate

【描述】

创建一个计算图

【声明】

```
RockXGraphInfo* RockXBuilderGraphCreate(const char* name);
```

【输入】

参数	输入/输出	类型	说明
name	输入	const char*	计算图名称

【返回值】

RockXGraphInfo*：计算图信息指针

【说明】

无

RockXBuilderGraphDestroy

【描述】

销毁一个计算图

【声明】

```
int RockXBuilderGraphDestroy(RockXGraphInfo* graph);
```

【输入】

参数	输入/输出	类型	说明
graph	输入	RockXGraphInfo*	计算图信息

【返回值】

int：0表示成功，其他为失败

【说明】

无

结构体定义

结构体	描述
RockXInputsInfo	输入列表
RockXOutputsInfo	输出列表
RockXSharesInfo	共享数据列表
RockXParamsInfo	配置列表
RockXNodesInfo	节点列表
RockXNodeInfo	节点信息
RockXGraphInfo	计算图信息

RockXNodeInfo

【描述】

节点信息，节点是模块的一个实例

【声明】

```
typedef struct RockXNodeInfo RockXNodeInfo;
```

【字段】

该结构体为不透明类型

【说明】

无

RockXGraphInfo

【描述】

计算图信息，计算图由多个节点组成

【声明】

```
typedef struct RockXNodeInfo RockXGraphInfo;
```

【字段】

该结构体为不透明类型

【说明】

无

工具函数

RockXPacket操作函数

函数	描述
RockXPacketSetInt	将数据包数据设置为一个整数数值
RockXPacketSetLong	将数据包数据设置为一个长整型数值
RockXPacketSetDouble	将数据包数据设置为一个双精度浮点数值
RockXPacketSetFloat	将数据包数据设置为一个浮点数值
RockXPacketSetString	将数据包数据设置为一个字符串
RockXPacketSetPointer	将数据包数据设置为一个指针
RockXPacketSetTensor	将数据包数据设置为一个张量
RockXPacketSetImage	将数据包数据设置为一个图像
RockXPacketShareData	数据包共享数据管理
RockXPacketMoveData	数据包数据管理转移

RockXPacketSetInt

【描述】

将数据包数据设置为一个整数数值

【声明】

```
RockXRetCode RockXPacketSetInt(RockXPacket* packet, int v);
```

【输入】

参数	输入/输出	类型	说明
packet	输入	RockXPacket*	数据包指针
v	输入	int	整数数值

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXPacketSetLong

【描述】

将数据包数据设置为一个长整型数值

【声明】

```
RockXRetCode RockXPacketSetLong(RockXPacket* packet, long v);
```

【输入】

参数	输入/输出	类型	说明
packet	输入	RockXPacket*	数据包指针
v	输入	long	长整型数值

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXPacketSetDouble

【描述】

将数据包数据设置为一个双精度浮点数值

【声明】

```
RockXRetCode RockXPacketSetDouble(RockXPacket* packet, double v);
```

【输入】

参数	输入/输出	类型	说明
packet	输入	RockXPacket*	数据包指针
v	输入	double	双精度浮点数

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXPacketSetFloat

【描述】

将数据包数据设置为一个浮点数值

【声明】

```
RockXRetCode RockXPacketSetFloat(RockXPacket* packet, float v);
```

【输入】

参数	输入/输出	类型	说明
packet	输入	RockXPacket*	数据包指针
v	输入	float	浮点数值

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXPacketSetString

【描述】

将数据包数据设置为一个字符串

【声明】

```
RockXRetCode RockXPacketSetString(RockXPacket* packet, const char* v, int needRelease);
```

【输入】

参数	输入/输出	类型	说明
packet	输入	RockXPacket*	数据包指针
v	输入	const char*	字符串
needRelease	输入	int	是否需要释放内存

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXPacketSetPointer

【描述】

将数据包数据设置为一个指针

【声明】

```
RockXRetCode RockXPacketSetPointer(RockXPacket* packet, void* v, int needRelease, const
char* typeName);
```

【输入】

参数	输入/输出	类型	说明
packet	输入	RockXPacket*	数据包指针
v	输入	void*	指针
needRelease	输入	int	是否需要释放内存
typeName	输入	const char*	指针类型名称

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXPacketSetTensor

【描述】

将数据包数据设置为一个张量

【声明】

```
RockXRetCode RockXPacketSetTensor(RockXPacket* packet, RockXTensor* tensor);
```

【输入】

参数	输入/输出	类型	说明
packet	输入	RockXPacket*	数据包指针
tensor	输入	RockXTensor*	张量指针

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXPacketSetImage

【描述】

将数据包数据设置为一个图像

【声明】

```
RockXRetCode RockXPacketSetImage(RockXPacket* packet, RockXImage* image);
```

【输入】

参数	输入/输出	类型	说明
packet	输入	RockXPacket*	数据包指针
image	输入	RockXImage*	图像指针

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXPacketShareData

【描述】

数据包共享数据管理

【声明】

```
RockXRetCode RockXPacketShareData(RockXPacket* packet, RockXPacket* sharedPacket);
```

【输入】

参数	输入/输出	类型	说明
packet	输入	RockXPacket*	数据包指针
sharedPacket	输入	RockXPacket*	共享数据包指针

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXPacketMoveData

【描述】

数据包数据管理转移

【声明】

```
RockXRetCode RockXPacketMoveData(RockXPacket* packet, RockXPacket* movedPacket);
```

【输入】

参数	输入/输出	类型	说明
packet	输入	RockXPacket*	数据包指针
movedPacket	输入	RockXPacket*	转移数据包指针

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXParam操作函数

函数	描述
RockXSetParamInt	将参数数据设置为一个整数数值
RockXSetParamLong	将参数数据设置为一个长整型数值
RockXSetParamFloat	将参数数据设置为一个浮点数值
RockXSetParamString	将参数数据设置为一个字符串
RockXSetParamPointer	将参数数据设置为一个指针
RockXParamCreate	
RockXParamDestroy	
RockXParamDup	
RockXParamDup2	

RockXSetParamInt

【描述】

将参数数据设置为一个整数数值

【声明】

```
RockXRetCode RockXSetParamInt(RockXHandle handle, const char* nodeName, const char*
paramName, int value);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	实例句柄
nodeName	输入	const char*	节点
paramName	输入	const char*	参数名
value	输入	int	数值

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXSetParamLong

【描述】

将参数数据设置为一个长整型数值

【声明】

```
RockXRetCode RockXSetParamLong(RockXHandle handle, const char* nodeName, const char*
paramName, long value);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	参数指针
nodeName	输入	const char*	节点
paramName	输入	const char*	参数名
value	输入	long	数值

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXSetParamFloat

【描述】

将参数数据设置为一个双精度浮点数值

【声明】

```
RockXRetCode RockXSetParamFloat(RockXHandle handle, const char* nodeName, const char*
paramName, float value);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	参数指针
nodeName	输入	const char*	节点
paramName	输入	const char*	参数名
value	输入	float	数值

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXSetParamString

【描述】

将参数数据设置为一个字符串

【声明】

```
RockXRetCode RockXSetParamString(RockXHandle handle, const char* nodeName, const char*
paramName, const char* value,
                                 int needRelease);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	参数指针
nodeName	输入	const char*	节点
paramName	输入	const char*	参数名
value	输入	const char*	数值
needRelease	输入	int	是否需要释放

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXSetParamPointer

【描述】

将参数数据设置为一个指针

【声明】

```
RockXRetCode RockXSetParamPointer(RockXHandle handle, const char* nodeName, const char*
paramName, void* value,
                                  int needRelease, RockXParamCallback* cb);
```

【输入】

参数	输入/输出	类型	说明
handle	输入	RockXHandle	参数指针
nodeName	输入	const char*	节点
paramName	输入	const char*	参数名
value	输入	void*	数值
needRelease	输入	int	是否需要释放
cb	输入	RockXParamCallback*	数值操作回调

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXParamCreate

【描述】

创建一个参数

【声明】

```
RockXParam* RockXParamCreate(const char* name);
```

【输入】

参数	输入/输出	类型	说明
name	输入	const char*	参数名称

【返回值】

RockXParam*：返回创建的参数指针

【说明】

此函数用于创建一个RockXParam参数对象，需要指定参数名称。当不再使用时，应调用RockXParamDestroy函数释放资源。

RockXParamDestroy

【描述】

销毁一个参数

【声明】

```
void RockXParamDestroy(RockXParam* param);
```

【输入】

参数	输入/输出	类型	说明
param	输入	RockXParam*	参数指针

【返回值】

无

【说明】

此函数用于销毁由RockXParamCreate创建的参数对象，释放相关资源。

RockXParamDup

【描述】

复制一个参数

【声明】

```
RockXParam* RockXParamDup(const RockXParam* param, int need_copy_value);
```

【输入】

参数	输入/输出	类型	说明
param	输入	const RockXParam*	源参数指针
need_copy_value	输入	int	是否需要复制值数据

【返回值】

RockXParam*：返回复制的参数指针

【说明】

此函数用于复制一个RockXParam参数对象。参数need_copy_value指定是否需要复制参数的值数据，为1表示复制，为0表示不复制。当不再使用时，应调用RockXParamDestroy函数释放资源。

RockXParamDup2

【描述】

将一个参数的内容复制到另一个参数

【声明】

```
RockXRetCode RockXParamDup2(const RockXParam* srcParam, RockXParam* dstParam, int
need_copy_value);
```

【输入】

参数	输入/输出	类型	说明
srcParam	输入	const RockXParam*	源参数指针
dstParam	输出	RockXParam*	目标参数指针
need_copy_value	输入	int	是否需要复制值数据

【返回值】

RockXRetCode：返回操作状态码

【说明】

此函数用于将一个参数的内容复制到另一个已经存在的参数。参数need_copy_value指定是否需要复制参数的值数据，为1表示复制，为0表示不复制。与RockXParamDup不同，此函数不创建新的参数对象，而是在已有的目标参数上进行修改。

RockXValue操作函数

函数	描述
RockXValueSetInt	将RockXValue设置为一个整数数值
RockXValueSetLong	将RockXValue设置为一个长整型数值
RockXValueSetFloat	将RockXValue设置为一个浮点数值
RockXValueSetDouble	将RockXValue设置为一个双精度浮点数值
RockXValueSetString	将RockXValue设置为一个字符串
RockXValueSetPointer	将RockXValue设置为一个指针
RockXValueSetArray	将RockXValue设置为一个数组
RockXValueGetInt	获取RockXValue中的整数数值
RockXValueGetLong	获取RockXValue中的长整型数值
RockXValueGetFloat	获取RockXValue中的浮点数值

函数	描述
RockXValueGetDouble	获取RockXValue中的双精度浮点数值
RockXValueGetString	获取RockXValue中的字符串
RockXValueGetPointer	获取RockXValue中的指针
RockXValueGetArray	获取RockXValue中的数组
RockXValueRelease	释放RockXValue

RockXValueSetInt

【描述】

将RockXValue设置为一个整数数值

【声明】

```
RockXRetCode RockXValueSetInt(RockXValue* value, int v);
```

【输入】

参数	输入/输出	类型	说明
value	输入	RockXValue*	值指针
v	输入	int	整数数值

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXValueSetLong

【描述】

将RockXValue设置为一个长整型数值

【声明】

```
RockXRetCode RockXValueSetLong(RockXValue* value, long v);
```

【输入】

参数	输入/输出	类型	说明
value	输入	RockXValue*	值指针

参数	输入/输出	类型	说明
v	输入	long	长整型数值

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXValueSetDouble

【描述】

将RockXValue设置为一个双精度浮点数值

【声明】

```
RockXRetCode RockXValueSetDouble(RockXValue* value, double v);
```

【输入】

参数	输入/输出	类型	说明
value	输入	RockXValue*	值指针
v	输入	double	双精度浮点数

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXValueSetFloat

【描述】

将RockXValue设置为一个浮点数值

【声明】

```
RockXRetCode RockXValueSetFloat(RockXValue* value, float v);
```

【输入】

参数	输入/输出	类型	说明
value	输入	RockXValue*	值指针

参数	输入/输出	类型	说明
v	输入	float	浮点数值

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXValueSetString

【描述】

将RockXValue设置为一个字符串

【声明】

```
RockXRetCode RockXValueSetString(RockXValue* value, const char* v, int needRelease);
```

【输入】

参数	输入/输出	类型	说明
value	输入	RockXValue*	值指针
v	输入	const char*	字符串
needRelease	输入	int	是否需要释放内存

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXValueSetPointer

【描述】

将RockXValue设置为一个指针

【声明】

```
RockXRetCode RockXValueSetPointer(RockXValue* value, void* v, int needRelease);
```

【输入】

参数	输入/输出	类型	说明
value	输入	RockXValue*	值指针
v	输入	void*	指针
needRelease	输入	int	是否需要释放内存

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXValueSetArray

【描述】

将RockXValue设置为一个数组

【声明】

```
RockXRetCode RockXValueSetArray(RockXValue* value, void* v, int count, int needRelease);
```

【输入】

参数	输入/输出	类型	说明
value	输入	RockXValue*	值指针
v	输入	void*	数组指针
count	输入	int	元素数量
needRelease	输入	int	是否需要释放内存

【返回值】

RockXRetCode：返回操作状态码

【说明】

无

RockXMem操作函数

函数	功能
RockXMemAlloc	分配内存
RockXMemFree	释放内存

函数	功能
RockXMemSync	同步内存
RockXMemFreeByVirtAddr	释放内存
RockXMemDumpAll	打印所有分配的内存信息

RockXMemAlloc

【描述】

分配内存

【声明】

```
int RockXMemAlloc(const char* tag, size_t size, RockXMemType type, RockXMemoryBuffer* mem);
```

【输入】

参数	输入/输出	类型	说明
tag	输入	const char*	内存标签
size	输入	size_t	分配大小
type	输入	RockXMemType	内存类型
mem	输出	RockXMemoryBuffer*	内存缓冲区

【返回值】

int：0表示成功，其他为失败

【说明】

无

RockXMemFree

【描述】

释放内存

【声明】

```
int RockXMemFree(RockXMemoryBuffer* mem);
```

【输入】

参数	输入/输出	类型	说明
mem	输入	RockXMemoryBuffer*	内存缓冲区

【返回值】

int：0表示成功，其他为失败

【说明】

无

RockXMemSync

【描述】

同步内存

【声明】

```
int RockXMemSync(RockXMemoryBuffer* mem);
```

【输入】

参数	输入/输出	类型	说明
mem	输入	RockXMemoryBuffer*	内存缓冲区

【返回值】

int：0表示成功，其他为失败

【说明】

无

RockXMemFreeByVirtAddr

【描述】

通过虚拟地址释放内存

【声明】

```
int RockXMemFreeByVirtAddr(void* virtAddr);
```

【输入】

参数	输入/输出	类型	说明
virtAddr	输入	void*	虚拟地址

【返回值】

int：0表示成功，其他为失败

【说明】

无

RockXMemDumpAll

【描述】

打印所有内存信息

【声明】

```
int RockXMemDumpAll();
```

【输入】

无

【返回值】

int：0表示成功，其他为失败

【说明】

无

RockXTensor操作函数

函数	功能
RockXTensorReadFromFile	从文件中读取张量
RockXTensorWrite	将张量写入文件
RockXTensorWriteTxt	将张量写入文本文件
RockXTensorSize	获取张量的内存大小
RockXTensorDump	打印张量信息
RockXTensorGetData	获取张量数据（按NCHW索引）
RockXTensorGetData1	获取张量数据（按一维索引）
RockXTensorGetDataInt8	获取张量INT8类型数据

RockXTensorReadFromFile

【描述】

从文件中读取张量数据

【声明】

```
RockXRetCode RockXTensorReadFromFile(const char* path, RockXDataType dtype, RockXTensor*
out_tensor);
```

【输入】

参数	输入/输出	类型	说明
path	输入	const char*	文件路径
dtype	输入	RockXDataType	数据类型
out_tensor	输出	RockXTensor*	输出张量

【返回值】

RockXRetCode：返回操作状态码

【说明】

此函数用于从指定路径的文件中读取张量数据，将数据读取到out_tensor结构中。需要指定读取的数据类型。

RockXTensorWrite

【描述】

将张量数据写入文件

【声明】

```
void RockXTensorWrite(const char* path, RockXTensor* tensor);
```

【输入】

参数	输入/输出	类型	说明
path	输入	const char*	文件路径
tensor	输入	RockXTensor*	张量

【返回值】

无

【说明】

此函数用于将张量数据以二进制形式写入指定路径的文件中。

RockXTensorWriteTxt

【描述】

将张量数据写入文本文件

【声明】

```
RockXRetCode RockXTensorWriteTxt(const char* path, RockXTensor* tensor);
```

【输入】

参数	输入/输出	类型	说明
path	输入	const char*	文件路径
tensor	输入	RockXTensor*	张量

【返回值】

RockXRetCode：返回操作状态码

【说明】

此函数用于将张量数据以文本形式写入指定路径的文件中，便于查看和分析。

RockXTensorSize

【描述】

获取张量的内存大小

【声明】

```
size_t RockXTensorSize(RockXTensor* tensor);
```

【输入】

参数	输入/输出	类型	说明
tensor	输入	RockXTensor*	张量

【返回值】

size_t：张量占用的内存大小（字节）

【说明】

此函数用于计算张量占用的内存大小，返回值为字节数。

RockXTensorDump

【描述】

打印张量信息

【声明】

```
void RockXTensorDump(RockXTensor* tensor);
```

【输入】

参数	输入/输出	类型	说明
tensor	输入	RockXTensor*	张量

【返回值】

无

【说明】

此函数用于打印张量的详细信息，包括形状、数据类型、量化信息等，主要用于调试。

RockXTensorGetData

【描述】

按NCHW索引获取张量数据

【声明】

```
RockXRetCode RockXTensorGetData(RockXTensor* tensor, int n, int c, int h, int w, float*
data);
```

【输入】

参数	输入/输出	类型	说明
tensor	输入	RockXTensor*	张量
n	输入	int	批次索引
c	输入	int	通道索引
h	输入	int	高度索引
w	输入	int	宽度索引
data	输出	float*	数据输出指针

【返回值】

RockXRetCode：返回操作状态码

【说明】

此函数用于从张量中按照NCHW格式的索引获取指定位置的数据，并以浮点数形式返回。适用于四维张量。

RockXTensorGetData1

【描述】

按一维索引获取张量数据

【声明】

```
RockXRetCode RockXTensorGetData1(RockXTensor* tensor, int index, float* data);
```

【输入】

参数	输入/输出	类型	说明
tensor	输入	RockXTensor*	张量
index	输入	int	一维索引
data	输出	float*	数据输出指针

【返回值】

RockXRetCode：返回操作状态码

【说明】

此函数用于从张量中按照一维索引获取指定位置的数据，并以浮点数形式返回。适用于任意维度的张量，简化了数据访问方式。

RockXTensorGetDataInt8

【描述】

按NCHW索引获取张量INT8类型数据

【声明】

```
RockXRetCode RockXTensorGetDataInt8(RockXTensor* tensor, int n, int c, int h, int w, int8_t*
data);
```

【输入】

参数	输入/输出	类型	说明
tensor	输入	RockXTensor*	张量
n	输入	int	批次索引
c	输入	int	通道索引

参数	输入/输出	类型	说明
h	输入	int	高度索引
w	输入	int	宽度索引
data	输出	int8_t*	数据输出指针

【返回值】

RockXRetCode：返回操作状态码

【说明】

此函数用于从张量中按照NCHW格式的索引获取指定位置的数据，并以INT8类型形式返回。主要用于量化后的张量数据获取。

RockXImage操作函数

函数	功能
RockXImageSize	根据图像宽度、高度和像素格式信息计算图像缓冲区内存大小
RockXImageAllocBuffer	根据图像宽度、高度和像素格式信息分配图像缓冲区内存
RockXImageFreeBuffer	释放图像缓冲区内存
RockXImageSyncBuffer	同步图像缓冲区内存，将缓存同步到内存
RockXImageWriteRaw	将图像原始数据写入文件

RockXImageSize

【描述】

根据图像宽度、高度和像素格式信息计算图像缓冲区内存大小

【声明】

```
size_t RockXImageSize(RockXImage* image);
```

【输入】

参数	输入/输出	类型	说明
image	输入	RockXImage*	图像对象

【返回值】

size_t：图像缓冲区内存大小（字节）

【说明】

此函数用于计算图像占用的内存大小，返回值为字节数。

RockXImageAllocBuffer

【描述】

根据图像宽度、高度和像素格式信息分配图像缓冲区内存

【声明】

```
RockXRetCode RockXImageAllocBuffer(RockXImage* image, RockXMemType type);
```

【输入】

参数	输入/输出	类型	说明
image	输入	RockXImage*	图像对象
type	输入	RockXMemType	内存类型

【返回值】

RockXRetCode：返回操作状态码

【说明】

此函数根据图像的宽度、高度和像素格式，为图像结构分配相应大小的内存缓冲区。

RockXImageFreeBuffer

【描述】

释放图像缓冲区内存

【声明】

```
RockXRetCode RockXImageFreeBuffer(RockXImage* image);
```

【输入】

参数	输入/输出	类型	说明
image	输入	RockXImage*	图像对象

【返回值】

RockXRetCode：返回操作状态码

【说明】

此函数用于释放由RockXImageAllocBuffer分配的图像缓冲区内存。

RockXImageSyncBuffer

【描述】

同步图像缓冲区内存，将缓存同步到内存

【声明】

```
RockXRetCode RockXImageSyncBuffer(RockXImage* image);
```

【输入】

参数	输入/输出	类型	说明
image	输入	RockXImage*	图像对象

【返回值】

RockXRetCode：返回操作状态码

【说明】

此函数用于同步图像缓冲区内存，确保缓存中的数据被写入到物理内存中，特别是在多核或GPU处理环境下使用。

RockXImageWriteRaw

【描述】

将图像原始数据写入文件

【声明】

```
RockXRetCode RockXImageWriteRaw(RockXImage* image, const char* path);
```

【输入】

参数	输入/输出	类型	说明
image	输入	RockXImage*	图像对象
path	输入	const char*	文件路径

【返回值】

RockXRetCode：返回操作状态码

【说明】

此函数将图像的原始数据直接写入到指定路径的文件中，不进行任何格式转换，主要用于调试和分析。
